Gen AI Prompt Log (Phased Learning Plan)

This log shows how AI was used as a tutor while learning Go from a Python background. Prompts were used to understand concepts, then code was implemented and validated manually.

Phase 1: Go Fundamentals (Python to Go mindset)

Entry 1: Basics and Syntax

Date: 2026-03-25
Prompt:
"I have a Python background. Teach me Go basics (variables, types, loops, conditionals, functions) by comparing each concept with Python and giving examples."
Learn Focus:
- Static typing vs Python dynamic typing
- for loops as the main Go loop
- Functions and multiple return values
Why:
- Foundation before writing any data pipeline code in Go.
Output Summary:
- Received side-by-side Python vs Go examples and beginner syntax explanations.
Validation:
- Rewrote simple Python-style snippets into Go.
- Compiled examples and fixed type errors manually.

Entry 2: Data Structures

Date: 2026-03-25
Prompt:
"Explain Go arrays, slices, maps, and structs with examples. Compare them with Python lists, dictionaries, and classes."
Learn Focus:
- Slices ([]T) for dynamic collections
- Maps for key-value indexing
- Structs for typed row models
Why:

- Core tools replacing most Pandas-like thinking in raw Go.
Output Summary:
- Received examples for converting list/dict/class patterns into Go slices/maps/structs.
Validation:
- Built row structs and used slices/maps in parsing and grouped summaries.

Entry 3: Error Handling

Date: 2026-03-25
Prompt:
"Teach me Go error handling patterns and how they differ from Python exceptions. Give real examples."
Learn Focus:
- if err != nil
- Explicit error returns instead of try/except
Why:
- Critical for file reading and safe data processing.
Output Summary:
- Learned value-and-error return style and handling at call sites.
Validation:
- Added error checks at I/O and model steps and tested invalid paths.

Phase 2: File Handling and Data Loading

Entry 4: Read CSV Files

Date: 2026-03-25
Prompt:
"Show me how to read and parse CSV files in Go using built-in packages. Then compare with pandas.read_csv."
Learn Focus:
- encoding/csv
- File I/O and row parsing
Why:
- First requirement for dataset workflow.
Output Summary:
- Reusable pattern for header normalization and field parsing.
Validation:
- Implemented CSV loaders and checked malformed-row skipping behavior.

Entry 5: Work with JSON Data

Date: 2026-03-25
Prompt:
"Teach me how to read, parse, and write JSON data in Go using structs. Compare with Python json and pandas."
Learn Focus:
- Struct tags, for example json:"stationId"
- Decode flow
Why:
- API and climate datasets are often JSON before conversion to CSV.
Output Summary:
- Learned nested JSON decoding and flattening into CSV rows.
Validation:
- Built JSON-to-CSV converter and verified output columns.

Phase 3: Data Cleaning and Preprocessing

Entry 6: Data Cleaning

Date: 2026-03-25
Prompt:
"Show me how to clean a dataset in Go: handling missing values, filtering rows, and transforming columns. Use CSV data examples."
Learn Focus:
- Manual cleaning logic
- Reusable parsing helper functions
Why:
- Base Go has no Pandas shortcut for cleaning.
Output Summary:
- Field-by-field parsing with required and optional values.
Validation:
- Tested missing-field and parse-error cases; confirmed dropped-row tracking.

Entry 7: Data Transformation Pipeline

Date: 2026-03-25
Prompt:
"Help me design a reusable data preprocessing pipeline in Go (like pandas pipelines), including filtering, mapping, and feature transformation."
Learn Focus:

- Modular function design
- Pipeline sequencing
Why:
- Maintainable structure for multi-step workflow.
Output Summary:
- Staged pipeline: load, clean, spatial match, model, report.
Validation:
- Confirmed each stage runs and merged output schema is correct.

Phase 4: Basic Data Analysis

Entry 8: Descriptive Statistics

Date: 2026-03-25
Prompt:
"Show me how to compute mean, median, variance, and standard deviation in Go. Compare with numpy."
Learn Focus:
- Math/stat helpers in Go
- Writing stats functions manually
Why:
- Core exploration before model training.
Output Summary:
- Manual stats calculations and summary table pattern.
Validation:
- Checked count/min/max/mean outputs against sample ranges.

Entry 9: Data Exploration

Date: 2026-03-25
Prompt:
"Help me perform exploratory data analysis (EDA) in Go, including summary stats and basic insights. Suggest libraries if needed."
Learn Focus:
- Coverage summaries by city/year/district
- Lightweight charting options
- Awareness of gonum and gota ecosystem
Why:
- Exploration and insight extraction from data.
Output Summary:

- Added text exploration report and SVG plots.
Validation:
- Confirmed report and plot files are generated every run.

Phase 5: Regression Model

Entry 10: Linear Regression
Date: 2026-03-25
Prompt:
"Teach me how to implement linear regression in Go from scratch, then using a library like gonum. Compare with scikit-learn."
Learn Focus:
- OLS intuition
- Matrix-based training workflow
Why:
- Core modeling component.
Output Summary:
- Implemented from-scratch OLS baseline.
Validation:
- Verified coefficients are finite and reproducible with fixed seed.

Entry 11: Model Evaluation

Date: 2026-03-25
Prompt:
"Show me how to evaluate a regression model in Go using metrics like RMSE and R-squared."
Learn Focus:
- RMSE, MAE, R^2
- Train/test split evaluation
Why:
- Objective model quality assessment.
Output Summary:
- Implemented metric and residual outputs.
Validation:
- Verified metrics in CLI/report and checked prediction residuals.

Phase 6: Project Integration

Entry 12: End-to-End Project

Date: 2026-03-25
Prompt:
"Guide me step-by-step to build a Go project that loads CSV, cleans data, performs EDA, trains regression, and evaluates model. Explain each step and structure like a real project."
Learn Focus:
- Real-world project layout
- End-to-end reproducible workflow
Why:
- Connect all project stages in one runnable pipeline.
Output Summary:
- Integrated CLI workflow with documented commands.
Validation:
- Ran prepare and model commands end-to-end and confirmed outputs.

Entry 13: Go vs Python for Data Science

Date: 2026-03-25
Prompt:
"Compare Go vs Python for data science in terms of performance, ecosystem, and use cases. When should I use Go?"
Learn Focus:
- Productivity vs performance trade-offs
- Best-fit use cases
Why:
- Helps justify technology choices in project write-up.
Output Summary:
- Python stronger for mature DS stack; Go stronger for performance and deployment simplicity.
Validation:
- Used comparison to frame method choices and limitations.

Responsible AI Use Statement

- AI was used as a tutor for concept learning, syntax translation, and debugging guidance.
- Outputs were validated through compile, run, and artifact checks.
- Final code decisions and integration were done manually.